LABORATORY EXPERIMENTAL PROTOCOL

Applied Massive Parallelism: CUDA Python, Thread Geometries, and Hardware Latency Pitfalls

Student Name	Anuarbek Zhasulan	**Student ID**	230103166
Session / Date	09.10.2026	**Assigned GPU**	Tesla T4

Submission note

This completed task sheet records the empirical values supplied from the executed Colab notebook. Student Name, Student ID, and Session / Date have been completed with the supplied submission details.

SECTION 1: ENVIRONMENT PROVISIONING & HARDWARE SETUP

Task 1.1 - Hardware Telemetry Extraction

Telemetry field	Recorded value
NVIDIA Driver Version	580.82.07
CUDA Version Supported by Driver	13.0
Allocated GPU Name	Tesla T4
Total Available VRAM	15360 MiB
Compute Capability Major.Minor	7.5
Max Threads Per Block	1024
Multiprocessor Count (SMs)	40
Max Block Dimensions	1024 x 1024 x 64
Warp Size	32

Hardware profile used for the benchmark and kernel experiments: NVIDIA Tesla T4, compute capability 7.5.

SECTION 2: THE LIE OF GPU SPEED & THE CROSSOVER EXPERIMENT

Task 2.2 - Benchmark Across Orders of Magnitude

Array Size N	CPU Time (ms)	GPU Total (ms)	GPU Kernel Only (ms)	Winner
100	0.077	1.314	0.091	CPU
1,000	0.257	0.601	0.058	CPU
10,000	2.330	0.499	0.056	GPU
100,000	23.868	0.661	0.065	GPU
1,000,000	260.233	4.612	0.123	GPU
5,000,000	N/A	12.996	0.251	CPU skipped
10,000,000	N/A	24.201	0.449	CPU skipped

Analysis Questions

Q1. What is the empirical Crossover Point N* on your machine?

The empirical crossover point on my machine is approximately **N = 10,000**. At N = 1,000, the CPU was still faster, but at N = 10,000 the GPU total execution time became lower than the CPU execution time.

Q2. At N = 100, why is GPU Total Time dramatically slower than CPU Time?

At N = 100, the workload is too small to compensate for GPU overhead. The kernel itself took only about 0.091 ms, while total GPU time was 1.314 ms. Most elapsed time was therefore spent on host-device memory transfers, device allocation, kernel launch, and synchronization rather than computation.

Observed crossover: CPU wins at N = 100 and 1,000; GPU total time first wins at N = 10,000.

SECTION 3: THREAD GEOMETRY & BOUNDARY MEMORY CORRUPTION

Task 3.1 - The Missing Guard Disaster Experiment

Observed behavior / console output:
The defective kernel completed without a visible CUDA error. The host copy succeeded, 1000 elements were returned, and the last element was 999.0. Although 1024 threads were launched for a 1000-element array, the out-of-bounds threads did not produce an observable crash in this run.

Interpretation: a GPU out-of-bounds access does not necessarily produce a Python IndexError. It may be silent or surface asynchronously, which is why an explicit boundary guard is required.

Task 3.2 - Spatial Thread Coordinate Inspector (N = 37)

Global ID	Block ID	Thread ID	Status
32	4	0	ACTIVE
33	4	1	ACTIVE
34	4	2	ACTIVE
35	4	3	ACTIVE
36	4	4	ACTIVE
37	4	5	IDLE (GUARDED)
38	4	6	IDLE (GUARDED)
39	4	7	IDLE (GUARDED)

Mandatory calculation	Result
Total Blocks Allocated	5
Total Hardware Threads Launched	40
Number of Active Working Threads	37
Number of Idle Guarded Threads	3

Calculation: ceil(37 / 8) = 5 blocks; 5 x 8 = 40 launched threads; 37 active + 3 guarded = 40.

SECTION 4: THE RACE CONDITION CRIME SCENE

Task 4.1 - The Naive Reduction Bug

Trial	Target Sum	Naive Kernel Output	Deviation / Lost Updates
1	50,000.0	2.0	49,998.0
2	50,000.0	2.0	49,998.0
3	50,000.0	2.0	49,998.0
4	50,000.0	2.0	49,998.0
5	50,000.0	2.0	49,998.0

Atomic Result and Timing

Measurement	Recorded result
Atomic Sum Output	50,000.0
Equals exactly 50,000.0?	Yes
Naive Kernel Time	0.636 ms
Atomic Kernel Time	0.532 ms

Performance penalty explanation

Atomic operations require synchronization when multiple threads update the same memory location. This reduces the amount of fully parallel execution and usually introduces additional overhead. In this single measurement, however, the atomic kernel was slightly faster than the naive kernel (0.532 ms vs 0.636 ms), most likely due to timing variation, GPU scheduling, warm-up effects, and measurement noise.

The naive result of 2.0 demonstrates lost updates: many threads read the same old value and overwrite one another's writes.

SECTION 5: 2D GRID IMAGE / MATRIX PROCESSING

1024 x 1024 Euclidean Radial Vignette

Verification field	Recorded result
Canvas dimensions	1024 x 1024
Threads per block	16 x 16 = 256
Blocks in X	64
Blocks in Y	64
Total Blocks	4096
Total Hardware Threads	1,048,576
Pixel at Center Point (512, 512)	1.000000
Pixel at Outer Corner Point (0, 0)	0.000000

Geometry verification: 1024 / 16 = 64 blocks along each axis; 64 x 64 = 4096 blocks. Each 16 x 16 block contains 256 threads, producing one thread per matrix element.

The center has maximum intensity (1.000000), while the outer corner is clamped to zero (0.000000), consistent with the radial vignette calculation.

Recorded output

Generated 2D matrix shape	(1024, 1024)
Blocks (X, Y)	(64, 64)
Center / Corner	1.000000 / 0.000000

SECTION 6: FINAL AI-IMMUNE VERIFICATION CHECKSUM

Final Submission Token & Integrity Check

The checksum was generated from the executed notebook using the Student ID supplied during that run, the Tesla T4 device identity, a sample of the computed 2D output, and the empirical crossover value N* = 10,000.

E63E11067B17B5C2

Student ID: 230103166
The token above is preserved exactly as produced by the user's executed validation cell.

Final Submission Checklist

Status	Submission requirement
[ ]	Notebook saved as CUDA_Lab01_230103166.ipynb
[ ]	All code cells executed sequentially with visible outputs
[ ]	Notebook downloaded as .ipynb and uploaded to the course portal
[x]	Student Name, Student ID, and Session / Date completed on this task sheet
[x]	Sections 1-6 empirical results entered
[x]	Verification token entered

Integrity statement: The numerical results in this document reproduce the values supplied from the user's CUDA Python laboratory execution. Student ID 230103166 and the submission details were supplied by the user.